

CSS Padding-Inline & Padding-Block

Complete Guide: Logical Properties for Responsive Design

CSS Guide

May 10, 2026

Contents

1	Introduction	2
1.1	What are Padding-Inline and Padding-Block?	2
1.2	Why Logical Properties Matter	2
1.2.1	Real-World Applications	2
1.2.2	Benefits	3
1.3	Browser Support	3
2	Core Concepts	4
2.1	Understanding Writing Directions	4
2.1.1	Physical vs Logical Axes	4
2.2	Padding-Inline Property	4
2.2.1	Syntax	4
2.2.2	Value Meanings	5
2.3	Padding-Block Property	5
2.3.1	Syntax	5
2.3.2	Value Meanings	5
3	Practical Examples	6
3.1	Example 1: Multilingual Button Component	6
3.2	Example 2: Card Component with Direction Adaptation	6
3.3	Example 3: Navigation Menu	7
3.4	Example 4: Form Input Styling	8
3.5	Example 5: Global Layout with Logical Properties	8
4	Advanced Techniques	10
4.1	Combining Logical Properties	10
4.1.1	Using Shorthand and Longhand	10
4.2	With CSS Variables	10
4.3	Direction-Aware Styling	11
5	Browser Support & Fallbacks	12
5.1	Progressive Enhancement	12
5.2	Feature Detection	12
6	Best Practices	13
6.1	Key Recommendations	13
6.1.1	1. Use Logical Properties for New Projects	13
6.1.2	2. Combine with Direction Attribute	13

6.1.3	3. Use for Consistent Spacing	13
7	Common Issues & Solutions	15
7.1	Issue 1: Unexpected Padding in RTL	15
7.2	Issue 2: Vertical Writing Support	15
8	Summary & Quick Reference	17
8.1	When to Use Logical Properties	17
8.2	Quick Reference	17
8.2.1	Property Mapping	17
8.2.2	Complete Example	17

Chapter 1

Introduction

1.1 What are Padding-Inline and Padding-Block?

The `padding-inline` and `padding-block` properties are CSS logical properties that control padding on the horizontal (inline) and vertical (block) axes respectively. Unlike traditional `padding-left`, `padding-right`, `padding-top`, and `padding-bottom`, these properties automatically adapt to the document's writing direction and text orientation.

```
/* Traditional padding: physical properties */
padding-left: 20px;
padding-right: 20px;
padding-top: 10px;
padding-bottom: 10px;

/* Logical properties: adapt to direction */
padding-inline: 20px;      /* left and right in LTR */
padding-block: 10px;      /* top and bottom */
```

Logical properties enable truly internationalized designs that automatically adapt to languages written right-to-left (Arabic, Hebrew) or top-to-bottom (Chinese, Japanese).

1.2 Why Logical Properties Matter

1.2.1 Real-World Applications

- **Internationalization (i18n):** Support RTL languages automatically
- **Arabic/Hebrew Support:** Content adapts without CSS changes
- **Asian Languages:** Support vertical writing modes
- **Flexible Layouts:** Write once, works in multiple languages
- **Modern Web Design:** Future-proof styling approach
- **Design Systems:** Build truly global component libraries

1.2.2 Benefits

- Automatic direction-aware layout
- No duplicate CSS for RTL variants
- Cleaner, more semantic CSS
- Better accessibility for multilingual content
- Simplified maintenance for global apps

1.3 Browser Support

Logical properties are well-supported in modern browsers:

Browser	Version	Status
Chrome/Edge	v69+	Full Support
Firefox	v68+	Full Support
Safari	v12+	Full Support
Opera	v56+	Full Support
IE 11	N/A	Not Supported

Chapter 2

Core Concepts

2.1 Understanding Writing Directions

2.1.1 Physical vs Logical Axes

Direction	Inline Axis	Block Axis
LTR (English)	Left→Right	Top→Bottom
RTL (Arabic)	Right→Left	Top→Bottom
Vertical (CJK)	Top→Bottom	Left→Right

2.2 Padding-Inline Property

The `padding-inline` property sets padding on both sides of the inline axis (left and right in LTR, right and left in RTL).

2.2.1 Syntax

```
/* Equal padding on inline start and end */
padding-inline: 20px;

/* Inline-start and inline-end */
padding-inline: 10px 20px;

/* Shorthand: same as padding-inline */
padding-inline-start: 10px;
padding-inline-end: 20px;

/* With units */
padding-inline: 1rem;
padding-inline: 1em 2em;

/* Percentage */
padding-inline: 10%;
```

2.2.2 Value Meanings

Syntax	LTR	RTL	Vertical
20px	L:20px, R:20px	R:20px, L:20px	T:20px, B:20px
10px 20px	L:10px, R:20px	R:10px, L:20px	T:10px, B:20px

2.3 Padding-Block Property

The `padding-block` property sets padding on both sides of the block axis (top and bottom in horizontal text, left and right in vertical text).

2.3.1 Syntax

```
/* Equal padding on block start and end */
padding-block: 10px;

/* Block-start and block-end */
padding-block: 15px 25px;

/* Shorthand versions */
padding-block-start: 15px;
padding-block-end: 25px;

/* With units */
padding-block: 1rem;
padding-block: 1em 2em;

/* Percentage */
padding-block: 5%;
```

2.3.2 Value Meanings

Syntax	Horizontal	Vertical LTR	Vertical RTL
10px	T:10px, B:10px	L:10px, R:10px	R:10px, L:10px
15px 25px	T:15px, B:25px	L:15px, R:25px	R:15px, L:25px

Chapter 3

Practical Examples

3.1 Example 1: Multilingual Button Component

```
/* Button with logical padding */
.button {
  padding-inline: 20px;
  padding-block: 10px;
  background: #667eea;
  color: white;
  border-radius: 4px;
  cursor: pointer;
}

/* English: padding-left: 20px, padding-right: 20px */
/* Arabic: padding-right: 20px, padding-left: 20px */
/* Automatically adapts! */

/* Alternative: Single logical property */
.button {
  padding: 10px 20px; /* Still works, but not logical */
}

/* Better with logical properties */
.button {
  padding-block: 10px;
  padding-inline: 20px;
}
```

3.2 Example 2: Card Component with Direction Adaptation

```
/* Card container */
.card {
  padding-block: 20px;
  padding-inline: 24px;
  background: white;
  border-radius: 8px;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
}
```



```
}

/* Title */
.card-title {
  padding-inline: 0;
  padding-block: 0 16px;
  font-size: 18px;
  font-weight: bold;
}

/* Content */
.card-content {
  padding-inline: 0;
  padding-block: 0;
}

/* Works perfectly in:
   - English (LTR)
   - Arabic (RTL)
   - Hebrew (RTL)
   - Chinese (Vertical)
*/
```

3.3 Example 3: Navigation Menu

```
/* Navigation list */
.nav-list {
  display: flex;
  list-style: none;
  margin: 0;
  padding: 0;
}

/* Navigation item */
.nav-item {
  padding-inline: 16px;
  padding-block: 12px;
}

/* Link */
.nav-link {
  text-decoration: none;
  color: #333;
}

.nav-link:hover {
  color: #667eea;
}

/* RTL: Automatically reverses inline padding */
/* LTR: padding-left: 16px, padding-right: 16px */
/* RTL: padding-right: 16px, padding-left: 16px */
```

3.4 Example 4: Form Input Styling

```
/* Input field */
input[type="text"],
input[type="email"],
textarea {
  padding-inline: 12px;
  padding-block: 8px;
  border: 1px solid #ddd;
  border-radius: 4px;
  font-size: 16px;
}

/* Focus state */
input:focus,
textarea:focus {
  outline: none;
  border-color: #667eea;
  box-shadow: 0 0 0 3px rgba(102, 126, 234, 0.1);
}

/* Works correctly in all languages
and writing directions */
```

3.5 Example 5: Global Layout with Logical Properties

```
/* Root element */
html {
  font-size: 16px;
}

/* Body */
body {
  margin: 0;
  padding-block: 20px;
  padding-inline: 20px;
  font-family: system-ui, sans-serif;
}

/* Container */
.container {
  max-width: 1200px;
  margin: 0 auto;
  padding-inline: 20px;
}

/* Section */
section {
  padding-block: 40px;
  padding-inline: 0;
}
```

```
/* Responsive padding */
@media (max-width: 768px) {
  body {
    padding-block: 15px;
    padding-inline: 15px;
  }

  section {
    padding-block: 30px;
  }
}
```

Chapter 4

Advanced Techniques

4.1 Combining Logical Properties

4.1.1 Using Shorthand and Longhand

```
/* Equivalent ways to set padding */

/* Method 1: Traditional */
.box {
    padding-top: 10px;
    padding-right: 20px;
    padding-bottom: 10px;
    padding-left: 20px;
}

/* Method 2: Traditional shorthand */
.box {
    padding: 10px 20px;
}

/* Method 3: Logical properties */
.box {
    padding-block: 10px;
    padding-inline: 20px;
}

/* Method 4: Mixed (not recommended) */
.box {
    padding-block: 10px;
    padding-inline-start: 20px;
    padding-inline-end: 20px;
}
```

4.2 With CSS Variables

```
/* Define spacing variables */
:root {
```

```
--spacing-xs: 4px;
--spacing-sm: 8px;
--spacing-md: 16px;
--spacing-lg: 24px;
--spacing-xl: 32px;
}

/* Use with logical properties */
.button {
  padding-block: var(--spacing-sm);
  padding-inline: var(--spacing-md);
}

.card {
  padding-block: var(--spacing-md);
  padding-inline: var(--spacing-lg);
}

.section {
  padding-block: var(--spacing-xl);
  padding-inline: 0;
}
```

4.3 Direction-Aware Styling

```
/* Respond to direction with @supports */
@supports (padding-inline: 0) {
  .modern-browser {
    padding-block: 20px;
    padding-inline: 30px;
  }
}

/* RTL-specific adjustments */
[dir="rtl"] .icon {
  margin-inline-start: 0;
  margin-inline-end: 10px;
}

[dir="ltr"] .icon {
  margin-inline-start: 10px;
  margin-inline-end: 0;
}
```

Chapter 5

Browser Support & Fallbacks

5.1 Progressive Enhancement

```
/* Fallback for older browsers */
.box {
    padding: 10px 20px;
}

/* Enhanced for modern browsers */
@supports (padding-inline: 0) {
    .box {
        padding-block: 10px;
        padding-inline: 20px;
    }
}
```

5.2 Feature Detection

```
/* Check for logical properties support */
@supports (padding-inline: 0) {
    /* Use logical properties */
    .component {
        padding-block: 20px;
        padding-inline: 30px;
    }
}

@supports not (padding-inline: 0) {
    /* Fallback to physical properties */
    .component {
        padding: 20px 30px;
    }
}
```

Chapter 6

Best Practices

6.1 Key Recommendations

6.1.1 1. Use Logical Properties for New Projects

```
/* Good: Future-proof */
.component {
  padding-block: 20px;
  padding-inline: 30px;
}

/* Acceptable: Works but not i18n-ready */
.component {
  padding: 20px 30px;
}
```

6.1.2 2. Combine with Direction Attribute

```
/* HTML */
/* <html dir="ltr"> for English */
/* <html dir="rtl"> for Arabic */

/* CSS automatically adapts */
.element {
  padding-block: 10px;
  padding-inline: 20px;
}
```

6.1.3 3. Use for Consistent Spacing

```
/* Define consistent spacing system */
.spacing-sm {
  padding-block: var(--space-sm);
  padding-inline: var(--space-sm);
}
```

```
.spacing-md {  
  padding-block: var(--space-md);  
  padding-inline: var(--space-md);  
}  
  
.spacing-lg {  
  padding-block: var(--space-lg);  
  padding-inline: var(--space-lg);  
}
```


Chapter 7

Common Issues & Solutions

7.1 Issue 1: Unexpected Padding in RTL

Problem: Padding appears on wrong side in RTL languages.

Cause: Using physical properties instead of logical.

Solution:

```
/* Wrong: Fixed sides */
.button {
    padding-left: 20px;
    padding-right: 10px;
}

/* Correct: Logical properties */
.button {
    padding-inline-start: 20px;
    padding-inline-end: 10px;
}

/* Works in both directions */
```

7.2 Issue 2: Vertical Writing Support

Problem: Layout breaks in vertical writing modes.

Solution:

```
/* Use logical properties */
.text {
    padding-block: 10px;
    padding-inline: 20px;
}

/* Set writing mode */
.vertical-text {
    writing-mode: vertical-rl;
    padding-block: 10px;
    padding-inline: 20px;
}
```

```
/* Automatically adapts to writing mode */
```

Chapter 8

Summary & Quick Reference

8.1 When to Use Logical Properties

Use logical properties when:

- Building multilingual/international apps
- Supporting RTL languages
- Supporting vertical writing modes
- Creating truly global components
- Future-proofing CSS code

8.2 Quick Reference

8.2.1 Property Mapping

```
/* Physical to Logical Mapping */

/* Horizontal padding */
padding-left: 10px; →      padding-inline-start: 10px;
padding-right: 10px; →     padding-inline-end: 10px;
padding-left + right →     padding-inline: 10px;

/* Vertical padding */
padding-top: 10px; →       padding-block-start: 10px;
padding-bottom: 10px; →    padding-block-end: 10px;
padding-top + bottom →     padding-block: 10px;

/* All padding */
padding: 10px 20px; →      padding-block: 10px; padding-inline: 20px;
```

8.2.2 Complete Example

```
/* Modern CSS with logical properties */
.container {
  padding-block: 40px;
  padding-inline: 20px;
}

.card {
  padding-block: 20px;
  padding-inline: 24px;
}

.button {
  padding-block: 12px;
  padding-inline: 24px;
}

/* Works perfectly in:
   English (LTR), Arabic (RTL), Chinese (Vertical)
   No duplicate CSS needed!
*/
```

Thank you for reading!

For more information, visit [MDN Web Docs](#) and [web.dev](#)